PukSquash Developer Manual

Court Availability Grid

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

The availability grid shows the state of every court and fixed time slot for a selected date. It provides the entry point for creating a booking or joining an Open booking. Laravel returns both the display state and action flags so the browser uses one set of availability rules.

BookCourtPage requests the selected date through bookingService. AvailabilityRepository fetches courts, active slots and confirmed bookings in one Data Connect operation. AvailabilityController orders the courts and slots, identifies past times and the current user's occupied times, then produces the cells rendered by AvailabilityGrid and explained by GridLegend.

1 1 Files and Responsibilities

resources/js/pages/BookCourtPage.vue — selected date and grid refresh

resources/js/components/AvailabilityGrid.vue and GridLegend.vue — display

resources/js/features/booking/bookingService.js — availability request

app/Http/Controllers/Api/AvailabilityController.php — state calculation

app/Repositories/AvailabilityRepository.php — GetAvailabilityData

1 2 API Contract

GET /api/availability?date=YYYY-MM-DD returns booking_window_days, courts, time_slots and cells. Each cell includes court_id, time_slot_id, state, label, can_book, can_join, is_mine and booking_id; occupied cells also include players.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Fetching the date snapshot

app/Repositories/AvailabilityRepository.php | source line 16 onward; blank and comment lines omitted

```
public function getForDate(string $bookingDate): array
    {
        $data = $this->client->execute('GetAvailabilityData', [
            'bookingDate' => $bookingDate,
        ]);
        $courts = array_map(
            fn (array $row) => CourtDTO::fromGraphQL($row),
            $data['courts'] ?? [],
        );
        $timeSlots = array_map(
            fn (array $row) => TimeSlotDTO::fromGraphQL($row),
            $data['timeSlots'] ?? [],
        );
        $bookings = array_map(
            fn (array $row) => BookingDTO::fromGraphQL($row),
            $data['bookings'] ?? [],
        );
        return [
            'courts'    => $courts,
            'timeSlots' => $timeSlots,
            'bookings'  => $bookings,
        ];
    }
}
```

Explanation

GetAvailabilityData returns the related data required to compute a complete grid. The repository maps the response into CourtDTO, TimeSlotDTO and BookingDTO objects. The controller then works with predictable property names instead of repeating raw GraphQL response handling.

Physical Code Explanation

1 The bookingDate variable selects the day. The operation name must match queries.gql exactly.

2 Each array_map converts a returned row to a typed DTO; missing response arrays default to empty arrays.

3 The returned snapshot keeps courts, timeSlots and bookings separate. Cells are derived by the controller, not persisted as a second availability table.

Times are interpreted in Africa/Johannesburg. The controller indexes confirmed bookings by court ID and slot ID, checks user participation, and disables free cells when the user is already booked in that time slot.

3 Choosing the occupied cell state

app/Http/Controllers/Api/AvailabilityController.php | source line 209 onward; blank and comment lines omitted

```
$state = match (true) {
    $booking->bookingType === 'PRIVATE_COACH'           => 'PRIVATE_COACH',
    $booking->bookingType === 'LOCKED'                  => 'LOCKED',
    $booking->bookingType === 'OPEN' && $playerCount < 2 => 'OPEN',
    default                                             => 'FULL',
};
$label = match ($state) {
    'OPEN'          => $nameA ? ($nameA . ' vs ?') : 'Open',
    'FULL'          => ($nameA && $nameB) ? ($nameA . ' vs ' . $nameB) : ($nameA ?:
'Full'),
    'LOCKED'        => $nameA ?: 'Locked',
    'PRIVATE_COACH' => ($nameA && $coachName)
        ? ($nameA . ' / Coach ' . $coachName)
        : ($nameA ?: ($coachName ? ('Coach ' . $coachName) : 'Coach')),
    default         => $nameA ?: $state,
};
```

Explanation

An existing booking determines the occupied state. A Private Coach or Locked booking remains closed to joining. An Open booking with fewer than two players is joinable only if the current caller is not already involved, the time has not started and the caller is free.

Physical Code Explanation

1 The match expression gives PRIVATE_COACH and LOCKED priority. OPEN is returned only for an Open booking with fewer than two players; other occupied cases become FULL.

2 The label uses participant and coach names when available and falls back to a readable state label.

3 Later in the method, canJoin also checks isMine, slotIsPast and userBusy. booking_id is provided for an eligible join; otherwise it is null.

A grid snapshot can become stale immediately after it is read. Recheck in the create and join endpoints. After a mutation or HTTP 409 conflict, refresh the selected date instead of silently showing the earlier grid.

4 Verification and Maintenance

Verification checklist

1 Use fixtures for AVAILABLE, OPEN, FULL, LOCKED, PRIVATE_COACH and UNAVAILABLE. Check both the legend colour and the action enabled on the cell.

2 Compare Court 2 and Court 10 ordering and check TimeSlot displayOrder.

3 Check today immediately before and after a slot starts, using South African time. Past cells must not offer booking or joining.

4 Book one court at a time slot, then check the other courts for that same user. Confirm overlapping participation is disabled.

5 Use two browser sessions: mutate a booking in one, refresh the other and verify the updated state.

Implementation boundaries

The payload advertises a fourteen-day booking window, but that field alone is not server enforcement of the window. The ordinary booking controller needs its own boundary check.

Live availability in this baseline means refreshed API snapshots. No WebSocket or Firestore realtime subscription is part of this feature.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).